Bash Cheat Sheet

A practical reference for everyday bash commands

Command History

```
!!            # Run the last command

CTRL+r        # Search bash history (hit CTRL+r multiple times)

touch foo.sh
chmod +x !$   # !$ is the last argument of the last command
```

Navigating Directories

```
pwd                      # Print current directory path
ls                       # List directories
ls -a|--all              # List directories including hidden
ls -l                    # List directories in long form
ls -l -h|--human-readable # List with human readable sizes
ls -t                    # List by modification time, newest first
stat foo.txt             # Size, created and modified timestamps
tree                     # List directory and file tree
tree -a                  # Including hidden
tree -d                  # Directories only
cd foo                   # Go to foo sub-directory
cd                       # Go to home directory
cd ~                     # Go to home directory
cd -                     # Go to last directory
pushd foo                # Go to foo and add previous to stack
popd                     # Go back to directory in stack
```

Creating Directories

```
mkdir foo                        # Create a directory
mkdir foo bar                    # Create multiple directories
mkdir -p|--parents foo/bar       # Create nested directory
mkdir -p|--parents {foo,bar}/baz # Create multiple nested directories

mktemp -d|--directory            # Create a temporary directory
```

Moving Directories

```
cp -R|--recursive foo bar                              # Copy directory
mv foo bar                                             # Move directory

rsync -z -v /foo /bar              # Copy, overwrites destination
rsync -a -z -v /foo /bar           # Copy, without overwriting
rsync -avz /foo user@host:/bar     # Local → remote
rsync -avz user@host:/foo /bar     # Remote → local
```

Deleting Directories

```
rmdir foo                        # Delete empty directory
rm -r|--recursive foo            # Delete directory including contents
rm -r -f|--force foo             # Force delete, never prompt
```

Creating Files

```
touch foo.txt          # Create file or update modified timestamp
```

```
touch foo.txt bar.txt  # Create multiple files
touch {foo,bar}.txt    # Create multiple files
touch test{1..3}       # Create test1, test2, test3
touch test{a..c}       # Create testa, testb, testc

mktemp                 # Create a temporary file
```

Standard Output / Error / Input

```
echo "foo" > bar.txt       # Overwrite file with content
echo "foo" >> bar.txt      # Append to file

ls exists 1> stdout.txt    # Redirect standard output
ls noexist 2> stderror.txt # Redirect standard error
ls 2>&1 > out.txt          # Redirect both stdout and stderr
ls > /dev/null             # Discard output

read foo                   # Read from stdin into variable
```

Moving & Deleting Files

```
cp foo.txt bar.txt                              # Copy file
mv foo.txt bar.txt                              # Move / rename file

rsync -z -v /foo.txt /bar                       # Fast copy if unchanged

rm foo.txt            # Delete file
rm -f|--force foo.txt # Force delete, never prompt
```

Reading Files

```
cat foo.txt            # Print all contents
less foo.txt           # Paginated view (g=top, G=bottom, /search)
head foo.txt           # Top 10 lines
tail foo.txt           # Bottom 10 lines
open foo.txt           # Open in default editor
wc foo.txt             # Count lines, words, characters
```

File Permissions

#	Permission	rwx	Binary
7	read, write and execute	rwx	111
6	read and write	rw-	110
5	read and execute	r-x	101
4	read only	r--	100
3	write and execute	-wx	011
2	write only	-w-	010
1	execute only	--x	001
0	none	---	000

For a directory, execute means you can enter it.

u = User g = Group o = Others a = All

```
ls -l /foo.sh           # List file permissions
chmod +100 foo.sh       # Add 1 to the user permission
chmod -100 foo.sh       # Subtract 1 from user permission
chmod u+x foo.sh        # Give user execute permission
chmod g+x foo.sh        # Give group execute permission
chmod u-x,g-x foo.sh    # Remove user & group execute
chmod a+x foo.sh        # Give everybody execute
chmod +x foo.sh         # Same as a+x
```

Finding Files

```
type wget                                   # Find the binary
which wget                                  # Find the binary
whereis wget                                # Binary, source, and man pages

# locate (uses index – fast)
updatedb                                    # Update the index
locate foo.txt                              # Find a file
locate --ignore-case foo                    # Case insensitive
locate f*.txt                               # Pattern match

# find (no index – slower but more powerful)
find /path -name foo.txt                    # Find a file
find /path -iname foo.txt                   # Case insensitive
find /path -name "*.txt"                    # All text files
find /path -name foo.txt -delete            # Find and delete
find /path -name "*.png" -exec pngquant {} # Run command on matches
find /path -type f -name foo.txt            # Files only
find /path -type d -name foo                # Directories only
find /path -type l -name foo.txt            # Symbolic links
find /path -type f -mtime +30               # Not modified in 30 days
find /path -type f -mtime +30 -delete       # Delete old files
```

Find in Files (grep)

```
grep 'foo' /bar.txt                          # Search in a file
grep 'foo' /bar -r|--recursive               # Search in directory
grep 'foo' /bar -R                           # Follow symbolic links
grep 'foo' /bar -l                           # Show only matching files
grep 'foo' /bar -L                           # Show non-matching files
grep 'Foo' /bar -i|--ignore-case             # Case insensitive
grep 'foo' /bar -x|--line-regexp             # Match entire line
grep 'foo' /bar -C 1                         # Context lines around match
grep 'foo' /bar -v|--invert-match            # Non-matching lines
grep 'foo' /bar -c|--count                   # Count matching lines
grep 'foo' /bar -n|--line-number             # Show line numbers
grep 'foo' /bar --colour                     # Colour output
grep 'foo\|bar' /baz -R                     # OR search
grep -E 'foo|bar' /baz -R                    # Extended regex
egrep 'foo|bar' /baz -R                      # Same as -E
```

Replace in Files (sed)

```
sed 's/fox/bear/g' foo.txt               # Replace and print to console
sed 's/fox/bear/gi' foo.txt              # Case insensitive replace
sed 's/red fox/blue bear/g' foo.txt      # Multi-word replace
sed 's/fox/bear/g' foo.txt > bar.txt     # Save to another file
sed 's/fox/bear/g' foo.txt -i|--in-place # Overwrite original
```

Symbolic Links

```
ln -s|--symbolic foo bar            # Create link 'bar' → 'foo'
ln -s -f|--force foo bar            # Overwrite existing link
ls -l                               # Show where links point
```

Compressing Files

```
# zip – one or more files → *.zip
zip foo.zip /bar.txt                # Compress single file
zip foo.zip /bar.txt /baz.txt       # Multiple files
zip foo.zip /{bar,baz}.txt          # Brace expansion
zip -r|--recurse-paths foo.zip /bar # Compress directory

# gzip – single file → *.gz
gzip /bar.txt                       # Compress and delete original
gzip -k|--keep /bar.txt             # Keep original

# tar – combine + optional compress
tar -czf foo.tgz /bar.txt /baz.txt  # Create .tgz
```

```
tar -czf foo.tgz /{bar,baz}.txt     # Brace expansion
tar -czf foo.tgz /bar               # Compress directory
```

Decompressing Files

```
unzip foo.zip                       # Unzip into current directory

gunzip foo.gz                       # Unzip and delete .gz
gunzip -k|--keep foo.gz             # Keep the .gz

tar -xzf foo.tar.gz                 # Extract .tar.gz
tar -xf foo.tar                     # Extract plain .tar
```

Disk & Memory Usage

```
df                      # List disks, size, used, available
df -h|--human-readable # Human readable sizes

du                      # Current dir + subdirs sizes
du /foo/bar             # Specific path
du -h                   # Human readable
du -d 0                 # Current directory size only

free                    # Memory usage
free -h|--human         # Human readable
free -h --si            # Power of 1000 instead of 1024
free -s 5               # Refresh every 5 seconds
```

Logs & System Debugging

```
journalctl -u nginx              # Logs for a specific service
journalctl -xe                   # Recent errors with explanations
journalctl -f                    # Follow logs in real time
journalctl -k                    # Kernel logs (systemd)

dmesg | tail                     # Recent kernel messages
tail -f /var/log/syslog          # Live system logs (Debian/Ubuntu)
tail -f /var/log/messages        # Live system logs (RHEL/CentOS)
```

Packages (apt)

```
apt update                   # Refresh repository index
apt search wget              # Search for a package
apt show wget                # Package information
apt list --all-versions wget # List all versions
apt install wget             # Install latest version
apt install wget=1.2.3       # Install specific version
apt remove wget              # Remove package
apt upgrade                  # Upgrade all upgradable packages
```

Shutdown and Reboot

```
shutdown                     # Shutdown in 1 minute
shutdown now "Cya later"     # Immediate shutdown
shutdown +5 "Cya later"      # Shutdown in 5 minutes

shutdown --reboot            # Reboot in 1 minute
shutdown -r now "Cya later"  # Immediate reboot
shutdown -r +5 "Cya later"   # Reboot in 5 minutes

shutdown -c                  # Cancel pending shutdown/reboot

reboot                       # Reboot now
reboot -f                    # Force reboot
```

Identifying & Controlling Processes

```
top                    # Interactive process list
htop                   # Better interactive process list
ps all                 # List all processes
pidof foo              # Return PID(s) of foo processes

CTRL+Z                 # Suspend foreground process
bg                     # Resume suspended process in background
fg                     # Bring last background process to foreground
fg 1                   # Bring specific job to foreground

kill PID               # Terminate process by PID
kill -9 PID            # Force kill
killall foo            # Kill all processes named foo
pkill -f pattern       # Kill by command pattern
```

Secure Shell (SSH)

```
ssh hostname                    # Connect as current user (port 22)
ssh -i foo.pem hostname         # Use identity file
ssh user@hostname               # Specify user
ssh user@hostname -p 8765       # Custom port
ssh ssh://user@hostname:8765    # URI style

# ~/.ssh/config example:
Host name
  User foo
  Hostname 127.0.0.1
  Port 8765
```

Secure Copy (scp)

```
scp foo.txt ubuntu@hostname:/home/ubuntu  # Local → remote
```

Bash Profile Tips

```
# Always run ls after cd
function cd {
  builtin cd "$@" && ls
}

# Prompt before overwriting
alias cp='cp --interactive'
alias mv='mv --interactive'
alias rm='rm --interactive'

# Human readable by default
alias df='df -h'
alias du='du -h'
```

Bash Scripting

Variables

```
#!/bin/bash

foo=123                # Initialize variable
declare -i foo=123     # Integer variable
declare -r foo=123     # Readonly variable
echo $foo              # Print variable
echo ${foo}_'bar'      # Concatenate
echo ${foo:-'default'} # Default if unset

export foo             # Export to child processes
unset foo              # Unset variable
```

Environment Variables

```
env            # List all environment variables
echo $PATH     # Print PATH
export FOO=Bar # Set environment variable
```

Functions

```
greet() {
  local world="World"
  echo "$1 $world"
}
greet "Hello"
greeting=$(greet "Hello")
```

Exit Codes

```
exit 0   # Success
exit 1   # Failure
echo $?  # Print last exit code
```

Conditional Operators

Numeric: -eq -ne -gt -ge -lt -le **String:** = == -z -n < >

File: -e (exists) **Boolean:** $var (true) !$var (false)

If Statements

```
if [[ $foo = 'bar' ]]; then
  echo 'one'
elif [[ $foo = 'baz' ]]; then
  echo 'two'
else
  echo 'other'
fi

# Inline
[[ $USER = 'rehan' ]] && echo 'yes' || echo 'no'
```

While Loop

```
declare -i counter=10
while [ $counter -gt 2 ]; do
  echo The counter is $counter
  counter=$((counter-1))
done
```

For Loops

```
for i in {0..10..2}; do
  echo "Index: $i"
done

for filename in file1 file2 file3; do
  echo "Content: " >> $filename
done

for filename in *; do
  echo "Content: " >> $filename
done
```

Case Statement

```
echo "What's the weather like tomorrow?"
read weather

case $weather in
  sunny|warm ) echo "Nice weather: $weather" ;;
  cloudy|cool ) echo "Not bad: $weather" ;;
  rainy|cold ) echo "Terrible: $weather" ;;
  * ) echo "Don't understand" ;;
esac
```